Chinook Data Engineering Pipeline

Databricks Medallion Architecture Implementation

March 28, 2026

Aravind Ravi | Aditya Rajesh Hasija | Vivek Nikam

Workspace: aravindworkspace | Catalog: workspace | Source: Azure SQL (damg7370-DB)

Table of Contents

1. Project Overview
2. Architecture & Data Flow
3. Source Table Inventory
4. Environment Setup (Connection Manager, Schemas, Volume)
5. Parameterization Design
6. Metadata Design
7. Notebook 01: Extract from Source
8. Notebook 02: Load Raw
9. Notebook 03: Raw to Bronze
10. Notebook 04: Bronze to Silver (Databricks DQX + Cleaning)
11. Notebook 05: Silver to Gold (Dimensional Model + SCD Type 2)
12. SCD Type 2 Demonstration
13. Job Orchestration
14. Dimensional Model
15. Data Quality Rules
16. SQL Scripts Reference
17. Git Repository
18. Validation Summary

1. Project Overview

This document presents a complete, metadata-driven data engineering pipeline built on Databricks using the Medallion Architecture pattern (Raw, Bronze, Silver, Gold). The pipeline ingests the Chinook music store dataset from Azure SQL Database, processes it through multiple quality-controlled layers, and produces a dimensional star schema suitable for analytics.

Source System: Azure SQL Database (damg7370-sqldbserver.database.windows.net / damg7370-DB)

Target Platform: Databricks (Unity Catalog, Serverless Compute)

Dataset: Chinook — 11 tables, 15,607 total rows (music store: albums, artists, customers, invoices, tracks)

Architecture: Medallion (Raw Parquet → Bronze Delta → Silver Delta → Gold Delta)

Key Features: Metadata-driven processing, DQX profiling and quarantine, SCD Type 2 for dim_customer, parameterized notebooks, Databricks Job orchestration with email notifications

2. Architecture & Data Flow

The pipeline follows a strict flow: Source → Raw (Parquet) → Bronze (Delta) → DQX Validation → Silver (Delta) → Gold (Delta). Key architectural rules enforced:

- Source data is never read directly into Bronze; it always passes through Raw first
- Raw snapshots are immutable — each run creates new timestamped folders, never overwrites
- Bronze is an exact Delta copy of Raw with zero transformations
- DQX profiling and validation occurs between Bronze and Silver
- Failed records are quarantined; only passed records proceed to Silver
- Silver applies DataFrame-based cleaning: trim, lowercase emails, null standardization, deduplication
- Gold builds a star schema using the mapping document, with SCD Type 2 on dim_customer
- fact_sales_customer_agg is aggregated from fact_sales, not directly from Silver

Object Structure

Object	Full Path
Raw Zone Schema	workspace.raw_zone
Raw Volume	workspace.raw_zone.chinook
Bronze Schema	workspace.bronze
Silver Schema	workspace.silver
Gold Schema	workspace.gold
Metadata Schema	workspace.metadata
Quarantine Schema	workspace.quarantine
Federation Catalog	chinook_azure_sql_catalog

3. Source Table Inventory

The Chinook dataset models a digital music store. All 11 tables reside in the chinook schema of the Azure SQL source database. The following inventory was validated before pipeline execution.

#	Table	Rows	Business Purpose	Gold Usage
1	Album	347	Album catalog with artist FK	dim_album
2	Artist	275	Music artist/band names	dim_artist
3	Customer	59	Store customers with contact info	dim_customer (SCD2)
4	Employee	8	Support representatives	dim_employee
5	Genre	25	Music genre classification	dim_genre
6	Invoice	412	Customer purchase invoices	fact_sales (join)
7	InvoiceLine	2,240	Line items per invoice	fact_sales (grain)
8	MediaType	5	Audio format (MPEG, AAC, etc.)	dim_media_type
9	Playlist	18	Curated playlists	Not in Gold
10	PlaylistTrack	8,715	Many-to-many: playlist to track	Not in Gold
11	Track	3,503	Track catalog with pricing	dim_track

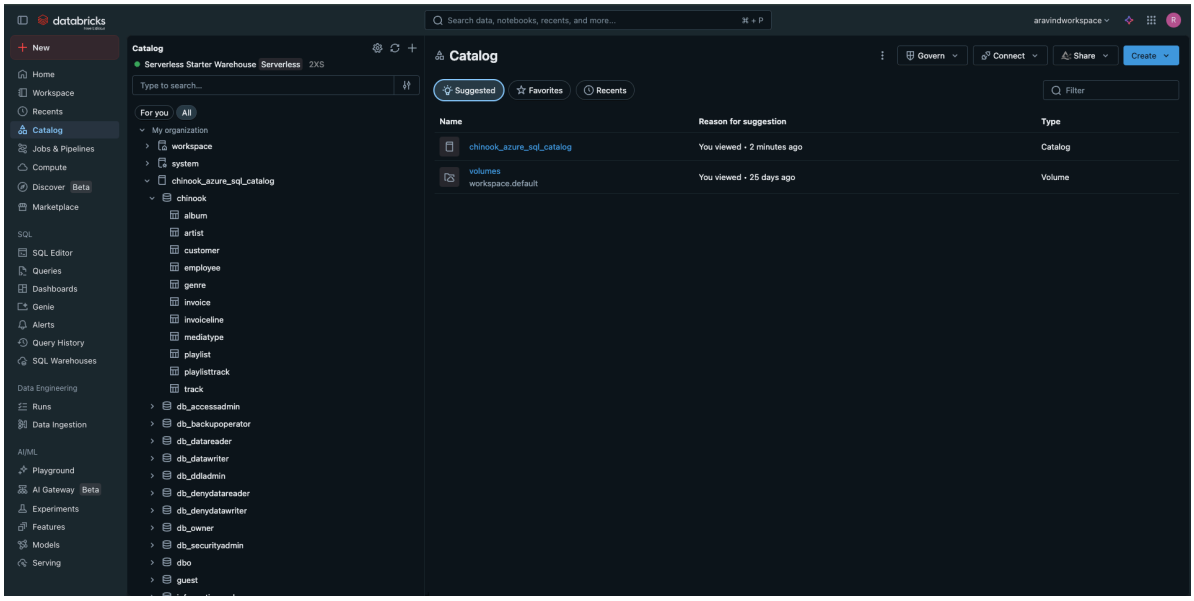
Total Source Rows: 15,607

Playlist and PlaylistTrack are ingested through all layers (Raw, Bronze, Silver) for completeness but are not used in the Gold dimensional model as they do not participate in the sales transaction grain.

4. Environment Setup

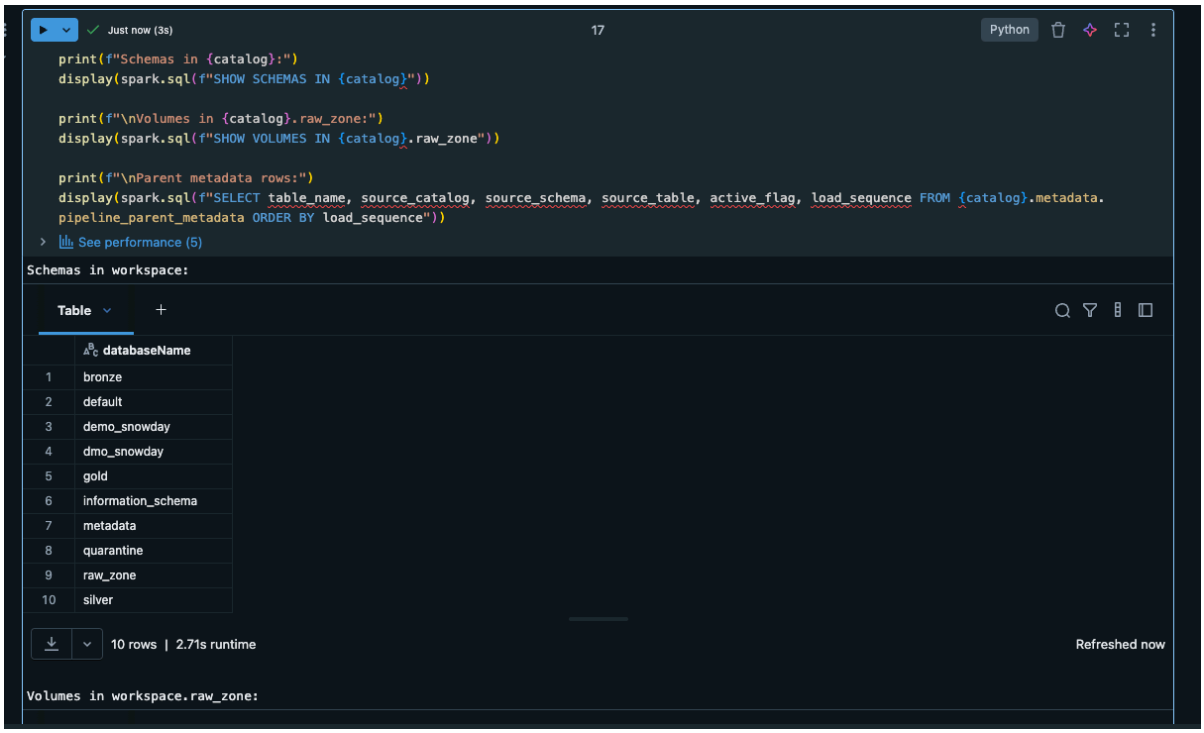
3.1 Databricks Connection Manager

A Lakehouse Federation connection was created to Azure SQL Database using the Databricks Connection Manager. The connection uses SQL Server authentication with read-only application intent. All 11 Chinook tables are accessible via the federation catalog chinook_azure_sql_catalog.chinook.<table>.



3.2 Unity Catalog Schemas

Six schemas were created under the workspace catalog to support the medallion layers, metadata tracking, and quarantine:



3.3 Raw Volume

A managed volume chinook was created inside raw_zone for storing immutable Parquet snapshots:

Volumes in workspace.raw_zone:

Table +

	database	volume_name
1	raw_zone	chinook

5. Parameterization Design

Every notebook is fully parameterized using Databricks widgets. No table names, catalog names, paths, or credentials are hardcoded. The four required parameters are defined at the top of every notebook and are also passed via the Databricks Job configuration.

5.1 Widget Parameters

Parameter	Default Value	Description
catalog_name	workspace	Unity Catalog name for all pipeline objects
schema_name	raw_zone	Default schema context for the pipeline
base_path	/Volumes/workspace/raw_zone/chinook	Root path for raw Parquet volume
table_name	all	Table filter (all = process every active table from metadata)
run_id	(auto-generated)	Unique identifier for each pipeline execution

5.2 Widget Code Block (Present in Every Notebook)

The following block appears at the top of notebooks 01–05:

```
# Widget parameters
dbutils.widgets.text("catalog_name", "workspace", "Catalog Name")
dbutils.widgets.text("schema_name", "raw_zone", "Schema Name")
dbutils.widgets.text("base_path", "/Volumes/workspace/raw_zone/chinook", "Base Path")
dbutils.widgets.text("table_name", "all", "Table Name")
dbutils.widgets.text("run_id", "", "Run ID (auto-generated if blank)")
```

5.3 Job-Level Parameter Configuration

All four parameters are configured at the Databricks Job level and automatically pushed to all five tasks. See Section 13.3 for the screenshot of the job parameter configuration panel.

6. Metadata Design

6.1 Parent Metadata Table

The pipeline_parent_metadata table drives all processing. Each row defines a source table with its federation reference, active flag, and load sequence. Notebooks read from this table to determine which tables to process — no table names are hardcoded.

Parent metadata rows:

Table

	table_name	source_catalog	source_schema	source_table	active_flag	load_sequence
1	album	chinook_azure_sql_catalog	chinook	album	Y	1
2	artist	chinook_azure_sql_catalog	chinook	artist	Y	2
3	customer	chinook_azure_sql_catalog	chinook	customer	Y	3
4	employee	chinook_azure_sql_catalog	chinook	employee	Y	4
5	genre	chinook_azure_sql_catalog	chinook	genre	Y	5
6	invoice	chinook_azure_sql_catalog	chinook	invoice	Y	6
7	invoiceline	chinook_azure_sql_catalog	chinook	invoiceline	Y	7
8	mediatype	chinook_azure_sql_catalog	chinook	mediatype	Y	8
9	playlist	chinook_azure_sql_catalog	chinook	playlist	Y	9
10	playlisttrack	chinook_azure_sql_catalog	chinook	playlisttrack	Y	10
11	track	chinook_azure_sql_catalog	chinook	track	Y	11

11 rows | 2.71s runtime Refreshed 6 minutes ago

6.2 Child Execution Metrics

The pipeline_child_metrics table logs every stage execution with row counts, status, file paths, and timestamps. This provides a complete audit trail for pipeline runs:

```
SELECT table_name, stage_name, status, source_row_count, target_row_count, file_location
FROM workspace.metadata.pipeline_child_metrics
ORDER BY created_date DESC
LIMIT 20
```

Table

	table_name	stage_name	status	source_row_count	target_row_count	file_location
1	track	bronze_to_silver	success	3503	3503	workspace.silver.track
2	playlisttrack	bronze_to_silver	success	8715	8715	workspace.silver.playlisttrack
3	playlist	bronze_to_silver	success	18	18	workspace.silver.playlist
4	mediatype	bronze_to_silver	success	5	5	workspace.silver.mediatype
5	invoiceline	bronze_to_silver	success	2240	2240	workspace.silver.invoiceline
6	invoice	bronze_to_silver	success	412	412	workspace.silver.invoice
7	genre	bronze_to_silver	success	25	25	workspace.silver.genre
8	employee	bronze_to_silver	success	8	8	workspace.silver.employee

20 rows | 3.58s runtime Refreshed now

7. Notebook 01: Extract from Source

This notebook reads the active table list from parent metadata and extracts each table via Lakehouse Federation (chinook_azure_sql_catalog.chinook.<table>). Each extraction is logged to the child metrics table with source row counts.

Extraction Output

```
Extracting: chinook_azure_sql_catalog.chinook.genre
  ✓ genre: 25 rows extracted (5.1s)

Extracting: chinook_azure_sql_catalog.chinook.invoice
  ✓ invoice: 412 rows extracted (2.3s)

Extracting: chinook_azure_sql_catalog.chinook.invoiceline
  ✓ invoiceline: 2240 rows extracted (2.3s)

Extracting: chinook_azure_sql_catalog.chinook.mediatype
  ✓ mediatype: 5 rows extracted (2.9s)

Extracting: chinook_azure_sql_catalog.chinook.playlist
  ✓ playlist: 18 rows extracted (2.2s)

Extracting: chinook_azure_sql_catalog.chinook.playlisttrack
  ✓ playlisttrack: 8715 rows extracted (2.2s)

Extracting: chinook_azure_sql_catalog.chinook.track
  ✓ track: 3503 rows extracted (2.3s)
```

Extraction Summary

All 11 tables extracted successfully with 15,607 total rows. Zero failures:

3 minutes ago (1s) 9

```
import pandas as pd

summary_df = pd.DataFrame(extraction_results)
print(f"\n{'='*60}")
print(f"EXTRACTION SUMMARY — Run ID: {run_id}")
print(f"{'='*60}")
print(f"Total tables: {len(extraction_results)}")
print(f"Successful:   {sum(1 for r in extraction_results if r['status'] == 'success')}")
print(f"Failed:       {sum(1 for r in extraction_results if r['status'] == 'failed')}")
print(f"Total rows:   {sum(r['source_count'] for r in extraction_results):,}")
print(f"{'='*60}")

display(spark.createDataFrame(summary_df))
```

See performance (1)

summary_df: pandas.core.frame.DataFrame = [table_name: object, source_count: int64 ... 3 more fields]

```
============================================================
EXTRACTION SUMMARY — Run ID: 1bb32e4f
============================================================
Total tables: 11
Successful:   11
Failed:       0
Total rows:   15,607
============================================================
```

Table +

	table_name	source_count	status	duration	error
1	album	347	success	5.778542	null
2	artist	275	success	2.646258	null
3	customer	59	success	2.297358	null
4	employee	8	success	2.429867	null
5	genre	25	success	5.063931	null
6	invoice	412	success	2.319693	null
7	invoiceline	2240	success	2.292693	null
8	mediatype	5	success	2.890057	null
9	playlist	18	success	2.180757	null
10	playlisttrack	8715	success	2.236046	null
11	track	3503	success	2.266031	null

11 rows | 0.99s runtime

8. Notebook 02: Load Raw

This notebook writes extracted DataFrames as immutable Parquet snapshots into the raw volume. Each run creates a new timestamped folder path (/Volumes/workspace/raw_zone/chinook/<table>/YYYY/MM/DD/HHMMSS/) ensuring snapshots are never overwritten.

Active Tables from Metadata

```
3 minutes ago (2s)
df_metadata = spark.sql(f"""
    SELECT table_name, source_catalog, source_schema, source_table, file_name, primary_key_col
    FROM {catalog}.metadata.pipeline_parent_metadata
    WHERE active_flag = 'Y'
    ORDER BY load_sequence
""")

tables = df_metadata.collect()
print(f"Tables to load to raw: {len(tables)}")
See performance (1)
df_metadata: pyspark.sql.connect.dataframe.DataFrame = [table_name: string, source_catalog: string ... 4 more fields]
Tables to load to raw: 11
```

Parquet Write Output

```
df: pyspark.sql.connect.dataframe.DataFrame = [TrackId: integer, Name: string ... 7 more fields]
df_verify: pyspark.sql.connect.dataframe.DataFrame = [TrackId: integer, Name: string ... 7 more fields]

Writing: genre → /Volumes/workspace/raw_zone/chinook/genre/2026/03/28/053151/genre.parquet
  ✓ genre: source=25, raw=25 (4.8s)

Writing: invoice → /Volumes/workspace/raw_zone/chinook/invoice/2026/03/28/053151/invoice.parquet
  ✓ invoice: source=412, raw=412 (5.0s)

Writing: invoiceline → /Volumes/workspace/raw_zone/chinook/invoiceline/2026/03/28/053151/invoiceline.parquet
  ✓ invoiceline: source=2240, raw=2240 (4.4s)

Writing: mediatype → /Volumes/workspace/raw_zone/chinook/mediatype/2026/03/28/053151/mediatype.parquet
  ✓ mediatype: source=5, raw=5 (4.5s)

Writing: playlist → /Volumes/workspace/raw_zone/chinook/playlist/2026/03/28/053151/playlist.parquet
  ✓ playlist: source=18, raw=18 (4.0s)

Writing: playlisttrack → /Volumes/workspace/raw_zone/chinook/playlisttrack/2026/03/28/053151/playlisttrack.parquet
  ✓ playlisttrack: source=8715, raw=8715 (4.2s)

Writing: track → /Volumes/workspace/raw_zone/chinook/track/2026/03/28/053151/track.parquet
  ✓ track: source=3503, raw=3503 (5.6s)
```

Raw Load Summary

All 11 tables loaded with source and target counts matching exactly — required validation per the assignment:

```
summary_df: pandas.core.frame.DataFrame = [table_name: object, source_count: int64 ... 4 more fields]

============================================================
RAW LOAD SUMMARY — Run ID: 5fe134f9
============================================================
Total tables:    11
Successful:      11
Count mismatch:  0
Failed:          0
============================================================
```

Table +

	table_name	source_count	target_count	file_path	status	duration
1	album	347	347	/Volumes/workspace/raw_zone/chinook/album/2026/03/28/053151/album.parquet	success	7.455292
2	artist	275	275	/Volumes/workspace/raw_zone/chinook/artist/2026/03/28/053151/artist.parquet	success	4.575743
3	customer	59	59	/Volumes/workspace/raw_zone/chinook/customer/2026/03/28/053151/customer.parquet	success	4.513914
4	employee	8	8	/Volumes/workspace/raw_zone/chinook/employee/2026/03/28/053151/employee.parquet	success	4.755514
5	genre	25	25	/Volumes/workspace/raw_zone/chinook/genre/2026/03/28/053151/genre.parquet	success	4.803063
6	invoice	412	412	/Volumes/workspace/raw_zone/chinook/invoice/2026/03/28/053151/invoice.parquet	success	4.951452
7	invoiceline	2240	2240	/Volumes/workspace/raw_zone/chinook/invoiceline/2026/03/28/053151/invoiceline.parquet	success	4.41556
8	mediatype	5	5	/Volumes/workspace/raw_zone/chinook/mediatype/2026/03/28/053151/mediatype.parquet	success	4.542713
9	playlist	18	18	/Volumes/workspace/raw_zone/chinook/playlist/2026/03/28/053151/playlist.parquet	success	4.043675
10	playlisttrack	8715	8715	/Volumes/workspace/raw_zone/chinook/playlisttrack/2026/03/28/053151/playlisttrack.parquet	success	4.183409
11	track	3503	3503	/Volumes/workspace/raw_zone/chinook/track/2026/03/28/053151/track.parquet	success	5.648061

11 rows | 7.93s runtime

Volume Structure Verification

4. Verify Volume Structure

5 minutes ago (<1s)

```
# Show raw volume contents
for t in [r["table_name"] for r in load_results if r["status"] == "success"]:
    print(f"  /{t}/")
print(f"\nFull path example: {load_results[0]['file_path'] if load_results else 'N/A'}")
```

```
  /album/
  /artist/
  /customer/
  /employee/
  /genre/
  /invoice/
  /invoiceline/
  /mediatype/
  /playlist/
  /playlisttrack/
  /track/

Full path example: /Volumes/workspace/raw_zone/chinook/album/2026/03/28/053151/album.parquet
```

5 minutes ago (<1s)

```
# Store values for downstream
dbutils.jobs.taskValues.set(key="run_id", value=run_id)
dbutils.jobs.taskValues.set(key="ts_path", value=ts_path)
print(f"Task values set — run_id: {run_id}, ts_path: {ts_path}")
```

```
Task values set — run_id: 5fe134f9, ts_path: 2026/03/28/053151
```

9. Notebook 03: Raw to Bronze

This notebook reads the latest Parquet snapshots from Raw and writes exact Delta table copies to Bronze using overwrite mode. Zero transformations are applied — Bronze is an identical mirror of Raw in Delta format.

Raw to Bronze Processing

```
> df_raw: pyspark.sql.connect.dataframe.DataFrame = [TrackId: integer, Name: string ... 7 more fields]

Processing: album
  Raw path: dbfs:/Volumes/workspace/raw_zone/chinook/album/2026/03/28/053151/album.parquet
  ✓ raw=347, bronze=347 (7.4s)

Processing: artist
  Raw path: dbfs:/Volumes/workspace/raw_zone/chinook/artist/2026/03/28/053151/artist.parquet
  ✓ raw=275, bronze=275 (5.8s)

Processing: customer
  Raw path: dbfs:/Volumes/workspace/raw_zone/chinook/customer/2026/03/28/053151/customer.parquet
  ✓ raw=59, bronze=59 (6.4s)

Processing: employee
  Raw path: dbfs:/Volumes/workspace/raw_zone/chinook/employee/2026/03/28/053151/employee.parquet
  ✓ raw=8, bronze=8 (5.6s)

Processing: genre
  Raw path: dbfs:/Volumes/workspace/raw_zone/chinook/genre/2026/03/28/053151/genre.parquet
  ✓ raw=25, bronze=25 (7.0s)
```

Bronze Load Summary

All raw counts match bronze counts exactly with zero mismatches:

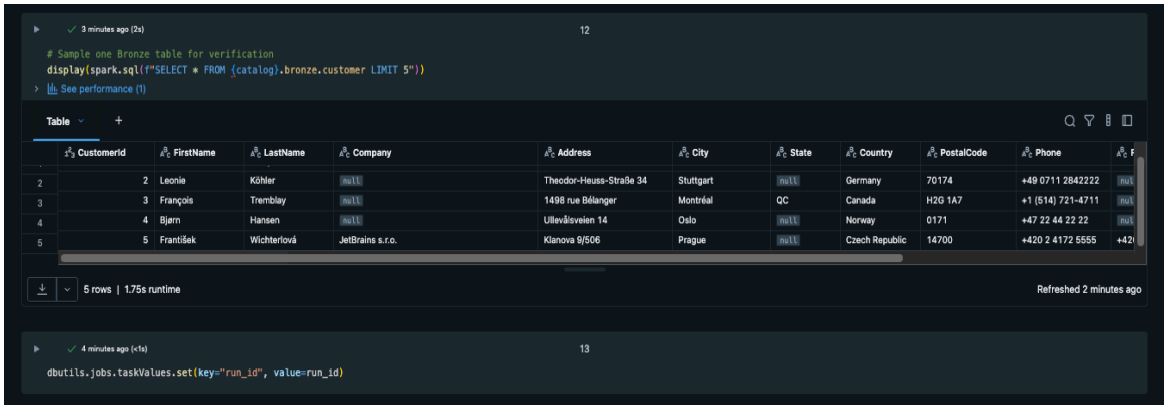
```
> summary_df: pandas.core.frame.DataFrame = [table_name: object, raw_count: int64 ... 3 more fields]

============================================================
RAW → BRONZE SUMMARY — Run ID: c48d845c
============================================================
Total:          11
Successful:     11
Mismatches:     0
Failed:         0
============================================================
```

Table +

	table_name	raw_count	bronze_count	status	duration
1	album	347	347	success	7.371369
2	artist	275	275	success	5.830712
3	customer	59	59	success	6.350319
4	employee	8	8	success	5.555288
5	genre	25	25	success	6.980686
6	invoice	412	412	success	5.757747
7	invoiceline	2240	2240	success	6.084658
8	mediatype	5	5	success	5.308194
9	playlist	18	18	success	6.049608
10	playlisttrack	8715	8715	success	5.481046
11	track	3503	3503	success	4.988598

Sample Bronze Table (Customer)

The Bronze customer table shows unmodified source data — nulls preserved, no cleaning applied:



3 minutes ago (2s) 12

```
# Sample one Bronze table for verification
display(spark.sql(f"SELECT * FROM {catalog}.bronze.customer LIMIT 5"))
```

See performance (1)

Table +

	CustomerId	FirstName	LastName	Company	Address	City	State	Country	PostalCode	Phone	F
2	2	Leonie	Köhler	null	Theodor-Heuss-Straße 34	Stuttgart	null	Germany	70174	+49 0711 2842222	nul
3	3	François	Tremblay	null	1498 rue Bélanger	Montréal	QC	Canada	H2G 1A7	+1 (514) 721-4711	nul
4	4	Bjørn	Hansen	null	Ullevålsveien 14	Oslo	null	Norway	0171	+47 22 44 22 22	nul
5	5	František	Wichterlová	JetBrains s.r.o.	Klanova 9/506	Prague	null	Czech Republic	14700	+420 2 4172 5555	+42

5 rows | 1.75s runtime Refreshed 2 minutes ago

4 minutes ago (<1s) 13

```
dbutils.jobs.taskValues.set(key="run_id", value=run_id)
```

10. Notebook 04: Bronze to Silver (Databricks DQX + Cleaning)

This is the most critical notebook. It uses the Databricks Labs DQX library (databricks-labs-dqx) for data profiling and quality validation. DQProfiler profiles each Bronze table to generate summary statistics. DQGenerator produces rule candidates from profiles. DQEngine applies quality checks using DQRowRule definitions with built-in check_funcs (is_not_null_and_not_empty, is_in_range). Records are split into valid and quarantined using apply_checks_and_split. DataFrame-based cleaning transformations are applied to valid records before writing to Silver.

10.1 Processing Output

Each table is profiled using DQProfiler (from databricks-labs-dqx) with sample_fraction=1.0. The profiler generates summary statistics including count, count_null, count_non_null, min, max, mean, stddev, and percentiles for each column:

```
====================================================
Processing: album (347 rows)
====================================================

  [DQX Profiler] Profiling album...
22:54:59  INFO [d.l.d.profiler.generator] ✅ Quality rules generation completed. Generated 5 rules.
  [DQX Profiler] Summary stats:
AlbumId:
  25%: 87
  50%: 174
  75%: 261
  count: 347
  count_non_null: 347
  count_null: 0
  max: 347
  mean: 174.0
  min: 1
  stddev: 100.31450543166726
ArtistId:
  25%: 58
```

10.2 Silver Summary

All 11 tables processed successfully using DQX apply_checks_and_split. The Chinook dataset is clean, so zero records were quarantined — but the DQX quarantine mechanism is fully implemented and operational:

```
================================================================
BRONZE → SILVER SUMMARY (Databricks DQX) - Run ID: 2d0ac997
================================================================
Total tables:      11
Successful:        11
Total quarantined: 0
================================================================
```

Table +

10.3 DQ Execution Log

27 data quality rules were executed across all tables using Databricks DQX check_funcs. Each rule logs total processed, passed, and failed counts:

6. View DQX Execution Log

✓ 06:54 PM (2s) 18

```
display(spark.sql(f"""
    SELECT table_name, rule_name, rule_description, total_processed, passed_count, failed_count
    FROM {catalog}.metadata.dq_execution_log
    WHERE run_id = '{run_id}'
    ORDER BY table_name, rule_name
"""))
```

See performance (1)

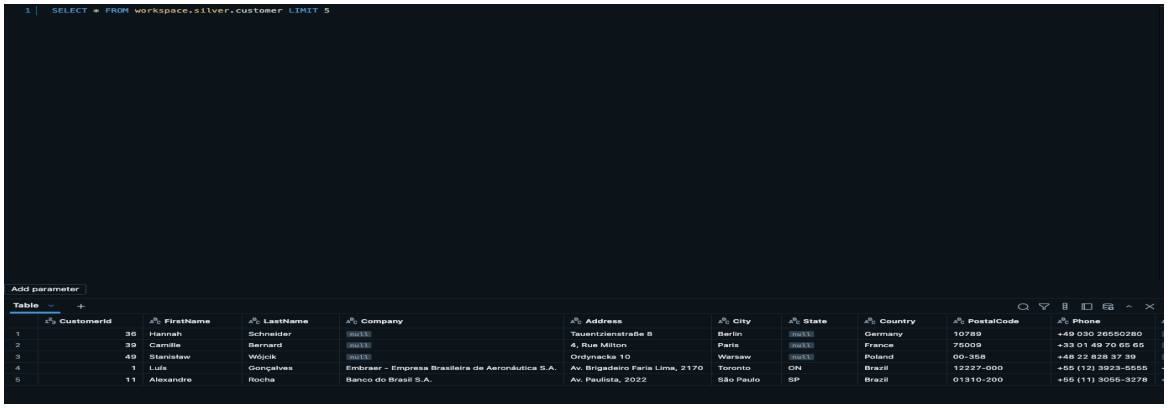
Table +

	table_name	rule_name	rule_description	total_processed	passed_count	failed_count
1	album	not_null_albumid	DQX check_funcs: AlbumId	347	347	0
2	album	not_null_title	DQX check_funcs: Title	347	347	0
3	artist	not_null_artistid	DQX check_funcs: ArtistId	275	275	0
4	customer	not_null_customerid	DQX check_funcs: CustomerId	59	59	0
5	customer	not_null_email	DQX check_funcs: Email	59	59	0
6	customer	not_null_firstname	DQX check_funcs: FirstName	59	59	0
7	customer	not_null_lastname	DQX check_funcs: LastName	59	59	0
8	employee	not_null_employeeid	DQX check_funcs: EmployeeId	8	8	0
9	employee	not_null_lastname	DQX check_funcs: LastName	8	8	0
10	genre	not_null_genreid	DQX check_funcs: GenreId	25	25	0
11	invoice	not_null_customerid	DQX check_funcs: CustomerId	412	412	0
12	invoice	not_null_invoicedate	DQX check_funcs: InvoiceDate	412	412	0
13	invoice	not_null_invoiceid	DQX check_funcs: InvoiceId	412	412	0
14	invoice	valid_total	DQX check_funcs: Total	412	412	0
15	invoiceline	not_null_invoiceid	DQX check_funcs: InvoiceId	2240	2240	0

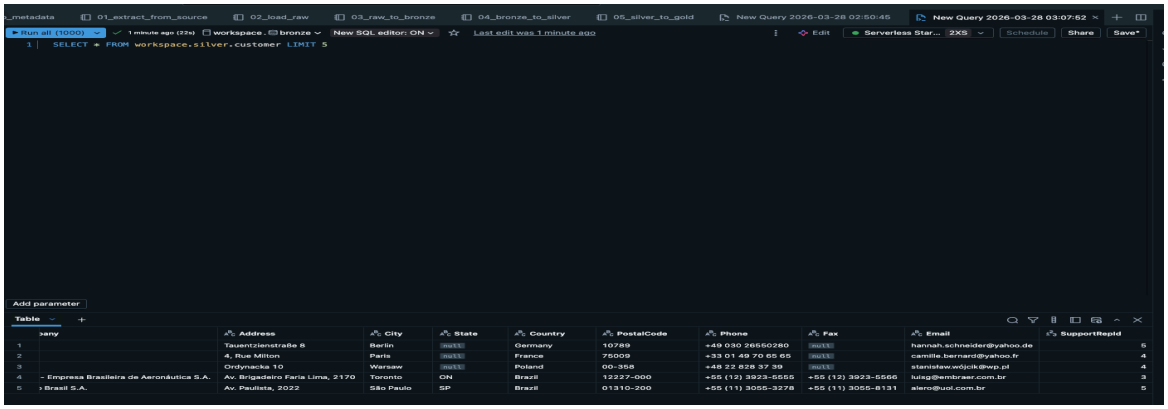
27 rows | 2.08s runtime Refreshed 5 minutes ago

10.4 Silver Cleaned Data

The Silver customer table shows cleaning transformations applied — emails lowercased, strings trimmed:



	CustomerId	FirstName	LastName	Company	Address	City	State	Country	PostalCode	Phone
1	36	Hannah	Schneider	null	Tauentzienstraße 8	Berlin	null	Germany	10789	+49 030 26550280
2	39	Camille	Bernard	null	4, Rue Milton	Paris	null	France	75009	+33 01 49 70 65 65
3	49	Stanisław	Wójcik	null	Ordynacka 10	Warsaw	null	Poland	00-358	+48 22 828 37 39
4	1	Luís	Gonçalves	Embraer - Empresa Brasileira de Aeronáutica S.A.	Av. Brigadeiro Faria Lima, 2170	Toronto	ON	Brazil	12227-000	+55 (12) 3923-5555
5	11	Alexandre	Rocha	Banco do Brasil S.A.	Av. Paulista, 2022	São Paulo	SP	Brazil	01310-200	+55 (11) 3055-3278



	pany	Address	City	State	Country	PostalCode	Phone	Fax	Email	SupportRepId
1		Tauentzienstraße 8	Berlin	null	Germany	10789	+49 030 26550280	null	hannah.schneider@yahoo.de	5
2		4, Rue Milton	Paris	null	France	75009	+33 01 49 70 65 65	null	camille.bernard@yahoo.fr	4
3		Ordynacka 10	Warsaw	null	Poland	00-358	+48 22 828 37 39	null	stanisaw.wójcik@wp.pl	4
4	- Empresa Brasileira de Aeronáutica S.A.	Av. Brigadeiro Faria Lima, 2170	Toronto	ON	Brazil	12227-000	+55 (12) 3923-5555	+55 (12) 3923-5566	luisg@embraer.com.br	3
5	Brasil S.A.	Av. Paulista, 2022	São Paulo	SP	Brazil	01310-200	+55 (11) 3055-3278	+55 (11) 3055-8131	aiero@uol.com.br	5

11. Notebook 05: Silver to Gold (Dimensional Model)

This notebook builds the complete star schema Gold layer from Silver tables using the mapping document. It implements SCD Type 2 for dim_customer with hash-based change detection.

11.1 Dimension Tables

dim_date — Generated date dimension covering 2009-01-01 to 2013-12-31 (1,826 rows):

1. dim_date (Generated — not from source)

01:50 AM (6s) 5

```python
import pandas as pd

# Generate date range covering all invoice dates
min_max = spark.sql(f"SELECT MIN(InvoiceDate) as min_dt, MAX(InvoiceDate) as max_dt FROM {catalog}.silver.invoice").collect()[0]
start_date = min_max.min_dt.date() if min_max.min_dt else date(2009, 1, 1)
end_date = min_max.max_dt.date() if min_max.max_dt else date(2025, 12, 31)

# Extend range to full years
start_date = date(start_date.year, 1, 1)
end_date = date(end_date.year, 12, 31)

date_range = pd.date_range(start=start_date, end=end_date, freq='D')
date_data = [{
    "date_key": int(d.strftime("%Y%m%d")),
    "full_date": d.date(),
    "year": d.year,
    "quarter": (d.month - 1) // 3 + 1,
    "month": d.month,
    "month_name": d.strftime("%B"),
    "day": d.day,
    "day_of_week": d.weekday() + 1,
    "day_name": d.strftime("%A"),
    "is_weekend": 1 if d.weekday() >= 5 else 0
} for d in date_range]

df_dim_date = spark.createDataFrame(date_data)
df_dim_date.write.mode("overwrite").saveAsTable(f"{catalog}.gold.dim_date")
print(f"dim_date: {df_dim_date.count()} rows ({start_date} to {end_date})")
```

See performance (3)

```
df_dim_date: pyspark.sql.connect.dataframe.DataFrame = [date_key: long, day: long ... 8 more fields]
dim_date: 1826 rows (2009-01-01 to 2013-12-31)
```

dim_artist (275 rows) and dim_album (347 rows):

2. dim_artist

01:50 AM (4s) 7

```
df_artist = spark.sql(f"""
    SELECT
        ArtistId AS artist_key,
        ArtistId AS artist_id,
        Name AS artist_name
    FROM {catalog}.silver.artist
""")
df_artist.write.mode("overwrite").saveAsTable(f"{catalog}.gold.dim_artist")
print(f"dim_artist: {df_artist.count()} rows")
```

See performance (2)

df_artist: pyspark.sql.connect.dataframe.DataFrame = [artist_key: integer, artist_id: integer ... 1 more field]

```
dim_artist: 275 rows
```

3. dim_album

+ Code + Text Genie Code

01:50 AM (4s) 9

```
df_album = spark.sql(f"""
    SELECT
        a.AlbumId AS album_key,
        a.AlbumId AS album_id,
        a.Title AS album_title,
        a.ArtistId AS artist_id,
        ar.Name AS artist_name
    FROM {catalog}.silver.album a
    LEFT JOIN {catalog}.silver.artist ar ON a.ArtistId = ar.ArtistId
""")
df_album.write.mode("overwrite").saveAsTable(f"{catalog}.gold.dim_album")
print(f"dim_album: {df_album.count()} rows")
```

See performance (2)

df_album: pyspark.sql.connect.dataframe.DataFrame = [album_key: integer, album_id: integer ... 3 more fields]

```
dim_album: 347 rows
```

dim_genre (25), dim_media_type (5), dim_employee (8):

4. dim_genre

01:50 AM (3s)

```
df_genre = spark.sql(f"""
    SELECT
        GenreId AS genre_key,
        GenreId AS genre_id,
        Name AS genre_name
    FROM {catalog}.silver.genre
""")
df_genre.write.mode("overwrite").saveAsTable(f"{catalog}.gold.dim_genre")
print(f"dim_genre: {df_genre.count()} rows")
```

See performance (2)

df_genre: pyspark.sql.connect.dataframe.DataFrame = [genre_key: integer, genre_id: integer ... 1 more field]

```
dim_genre: 25 rows
```

5. dim_media_type

01:50 AM (4s)

```
df_media = spark.sql(f"""
    SELECT
        MediaTypeId AS media_type_key,
        MediaTypeId AS media_type_id,
        Name AS media_type_name
    FROM {catalog}.silver.mediatype
""")
df_media.write.mode("overwrite").saveAsTable(f"{catalog}.gold.dim_media_type")
print(f"dim_media_type: {df_media.count()} rows")
```

See performance (2)

df_media: pyspark.sql.connect.dataframe.DataFrame = [media_type_key: integer, media_type_id: integer ... 1 more field]

```
dim_media_type: 5 rows
```

6. dim_employee

01:50 AM (4s)

```
df_employee = spark.sql(f"""
    SELECT
        EmployeeId AS employee_key,
        EmployeeId AS employee_id,
        FirstName AS first_name,
        LastName AS last_name,
        Title AS title,
        ReportsTo AS reports_to,
        BirthDate AS birth_date,
        HireDate AS hire_date,
        City AS city,
        State AS state,
        Country AS country,
        Email AS email
    FROM {catalog}.silver.employee
""")
df_employee.write.mode("overwrite").saveAsTable(f"{catalog}.gold.dim_employee")
print(f"dim_employee: {df_employee.count()} rows")
```

See performance (2)

df_employee: pyspark.sql.connect.dataframe.DataFrame = [employee_key: integer, employee_id: integer ... 10 more fields]

```
dim_employee: 8 rows
```

dim_track (3,503 rows):

7. dim_track

01:50 AM (4s)

```
df_track = spark.sql(f"""
    SELECT
        t.TrackId AS track_key,
        t.TrackId AS track_id,
        t.Name AS track_name,
        t.AlbumId AS album_id,
        t.MediaTypeId AS media_type_id,
        t.GenreId AS genre_id,
        t.Composer AS composer,
        t.Milliseconds AS milliseconds,
        t.Bytes AS bytes,
        t.UnitPrice AS unit_price
    FROM {catalog}.silver.track t
""")
df_track.write.mode("overwrite").saveAsTable(f"{catalog}.gold.dim_track")
print(f"dim_track: {df_track.count()} rows")
```

See performance (2)

```
df_track: pyspark.sql.connect.dataframe.DataFrame = [track_key: integer, track_id: integer ... 8 more fields]
dim_track: 3503 rows
```

11.2 dim_customer — SCD Type 2 (Initial Load)

Initial load: 59 customers, all with is_current=true and effective_end_date=9999-12-31:

01:50 AM (5s)

```
total_customer_rows, scd_changes = build_dim_customer_scd2(catalog)
print(f"\ndim_customer: {total_customer_rows} total rows, {scd_changes} SCD Type 2 changes")
```

See performance (3)

```
dim_customer does not exist — performing initial load

dim_customer: 59 total rows, 0 SCD Type 2 changes
```

11.3 Fact Tables

fact_sales at invoice line grain (2,240 rows) and fact_sales_customer_agg (59 rows) aggregated from fact_sales:

10. fact_sales (Invoice Line Grain)

01:50 AM (7s)

```
df_fact_sales = spark.sql(f"""
    SELECT
        il.InvoiceLineId AS sales_line_key,
        il.InvoiceId AS invoice_id,
        il.TrackId AS track_key,
        i.CustomerId AS customer_id,
        dc.customer_key AS customer_key,
        t.AlbumId AS album_key,
        t.GenreId AS genre_key,
        t.MediaTypeId AS media_type_key,
        CAST(DATE_FORMAT(i.InvoiceDate, 'yyyyMMdd') AS INT) AS date_key,
        i.InvoiceDate AS invoice_date,
        il.Quantity AS quantity,
        il.UnitPrice AS unit_price,
        CAST(il.Quantity * il.UnitPrice AS DECIMAL(10,2)) AS line_amount,
        i.Total AS invoice_total,
        e.EmployeeId AS employee_key
    FROM {catalog}.silver.invoiceline il
    JOIN {catalog}.silver.invoice i ON il.InvoiceId = i.InvoiceId
    JOIN {catalog}.silver.customer c ON i.CustomerId = c.CustomerId
    LEFT JOIN {catalog}.gold.dim_customer dc
        ON c.CustomerId = dc.customer_id AND dc.is_current = true
    LEFT JOIN {catalog}.silver.track t ON il.TrackId = t.TrackId
    LEFT JOIN {catalog}.silver.employee e ON c.SupportRepId = e.EmployeeId
""")

df_fact_sales.write.mode("overwrite").saveAsTable(f"{catalog}.gold.fact_sales")
print(f"fact_sales: {df_fact_sales.count()} rows")
```

See performance (2)

```
df_fact_sales: pyspark.sql.connect.dataframe.DataFrame = [sales_line_key: integer, invoice_id: integer ... 13 more fields]
fact_sales: 2240 rows
```

11. fact_sales_customer_agg (Aggregated from fact_sales — NOT from Silver)

01:50 AM (5s)

```
df_agg = spark.sql(f"""
    SELECT
        fs.customer_key,
        fs.customer_id,
        COUNT(DISTINCT fs.invoice_id)   AS total_invoices,
        COUNT(*)                        AS total_items_purchased,
        SUM(fs.line_amount)             AS total_sales_amount,
        MIN(fs.invoice_date)            AS first_purchase_date,
        MAX(fs.invoice_date)            AS last_purchase_date,
        COUNT(DISTINCT fs.track_key)    AS distinct_tracks_purchased,
        COUNT(DISTINCT fs.genre_key)    AS distinct_genres
    FROM {catalog}.gold.fact_sales fs
    GROUP BY fs.customer_key, fs.customer_id
""")

df_agg.write.mode("overwrite").saveAsTable(f"{catalog}.gold.fact_sales_customer_agg")
print(f"fact_sales_customer_agg: {df_agg.count()} rows")
```

See performance (2)

```
df_agg: pyspark.sql.connect.dataframe.DataFrame = [customer_key: long, customer_id: integer ... 7 more fields]
fact_sales_customer_agg: 59 rows
```

01:50 AM (1s) 33

```
# Sample aggregate — top customers by sales
display(spark.sql(f"SELECT * FROM {catalog}.gold.fact_sales_customer_agg ORDER BY total_sales_amount DESC LIMIT 10"))
```

See performance (1)

Table

	customer_key	customer_id	total_invoices	total_items_purchased	total_sales_amount	first_purchase_date	last_purchase_date	distinct_tracks_purchased	distinct_genres
1	34	6	7	38	49.62	2009-07-11T00:00:00.000+00:00	2013-11-13T00:00:00.000+00:00	38	9
2	50	26	7	38	47.62	2009-11-07T00:00:00.000+00:00	2013-04-06T00:00:00.000+00:00	38	7
3	11	57	7	38	46.62	2009-04-04T00:00:00.000+00:00	2012-10-14T00:00:00.000+00:00	38	12
4	15	45	7	38	45.62	2010-01-06T00:00:00.000+00:00	2013-07-20T00:00:00.000+00:00	38	11
5	31	46	7	38	45.62	2009-02-03T00:00:00.000+00:00	2013-11-04T00:00:00.000+00:00	38	8
6	8	24	7	38	43.62	2010-02-08T00:00:00.000+00:00	2013-08-20T00:00:00.000+00:00	38	10
7	24	37	7	38	43.62	2009-01-19T00:00:00.000+00:00	2013-06-03T00:00:00.000+00:00	38	10
8	32	28	7	38	43.62	2009-11-07T00:00:00.000+00:00	2013-05-19T00:00:00.000+00:00	38	6
9	35	7	7	38	42.62	2009-12-08T00:00:00.000+00:00	2013-06-19T00:00:00.000+00:00	38	9
10	57	25	7	38	42.62	2009-03-06T00:00:00.000+00:00	2013-12-05T00:00:00.000+00:00	38	8

10 rows | 1.24s runtime Refreshed 10 minutes ago

11.4 Gold Summary & Validation

12. Gold Summary and Validation

01:50 AM (6s)

```python
gold_tables = [
    "dim_date", "dim_artist", "dim_album", "dim_genre",
    "dim_media_type", "dim_employee", "dim_track", "dim_customer",
    "fact_sales", "fact_sales_customer_agg"
]

print(f"{'='*50}")
print(f"GOLD LAYER SUMMARY — Run ID: {run_id}")
print(f"{'='*50}")

for tbl in gold_tables:
    cnt = spark.sql(f"SELECT COUNT(*) as cnt FROM {catalog}.gold.{tbl}").collect()[0].cnt
    print(f"  {tbl:35s} {cnt:>10,} rows")
```

See performance (10)

```
==================================================
GOLD LAYER SUMMARY — Run ID: 2c4cfd97
==================================================
  dim_date                                1,826 rows
  dim_artist                                275 rows
  dim_album                                 347 rows
  dim_genre                                  25 rows
  dim_media_type                              5 rows
  dim_employee                                8 rows
  dim_track                               3,503 rows
  dim_customer                               59 rows
  fact_sales                              2,240 rows
  fact_sales_customer_agg                    59 rows
```

11.5 Dimensional Integrity Checks

Zero orphan customer keys in fact_sales. Aggregate totals match: fact_sales = 2328.60, fact_sales_customer_agg = 2328.60:

13. Dimensional Integrity Checks

01:50 AM (2s) 31

```python
# Check: every fact row joins to a customer dimension
orphan_customers = spark.sql(f"""
    SELECT COUNT(*) as orphans
    FROM {catalog}.gold.fact_sales fs
    LEFT JOIN {catalog}.gold.dim_customer dc ON fs.customer_key = dc.customer_key
    WHERE dc.customer_key IS NULL
""").collect()[0].orphans
print(f"Orphan customer keys in fact_sales: {orphan_customers}")

# Check: aggregate fact totals match fact_sales rollup
agg_total = spark.sql(f"SELECT SUM(total_sales_amount) as total FROM {catalog}.gold.fact_sales_customer_agg").collect()[0].total
fact_total = spark.sql(f"SELECT SUM(line_amount) as total FROM {catalog}.gold.fact_sales").collect()[0].total
print(f"fact_sales total:          {fact_total}")
print(f"fact_sales_cust_agg total: {agg_total}")
print(f"Match: {'✓' if abs(float(fact_total or 0) - float(agg_total or 0)) < 0.01 else '✗'}")
```

See performance (3)

```
Orphan customer keys in fact_sales: 0
fact_sales total:          2328.60
fact_sales_cust_agg total: 2328.60
Match: ✓
```

01:50 AM (2s) 32

```python
# Sample fact_sales
display(spark.sql(f"SELECT * FROM {catalog}.gold.fact_sales LIMIT 10"))
```

See performance (1)

Table

	sales_line_key	invoice_id	track_key	customer_id	customer_key	album_key	genre_key	media_type_key	date_key	invoice_date	quantity	unit_price	line_amount	inv
1	15	4	54	14	27	7	1	1	20090106	2009-01-06T00:00:00.000+00:00	1	0.99	0.99	
2	54	11	292	52	39	26	8	1	20090206	2009-02-06T00:00:00.000+00:00	1	0.99	0.99	
3	80	16	472	21	38	39	4	1	20090305	2009-03-05T00:00:00.000+00:00	1	0.99	0.99	
4	87	17	496	25	57	40	1	1	20090306	2009-03-06T00:00:00.000+00:00	1	0.99	0.99	
5	133	25	774	10	46	61	1	1	20090409	2009-04-09T00:00:00.000+00:00	1	0.99	0.99	
6	136	26	795	19	28	63	1	1	20090414	2009-04-14T00:00:00.000+00:00	1	0.99	0.99	
7	143	26	858	19	28	69	7	1	20090414	2009-04-14T00:00:00.000+00:00	1	0.99	0.99	
8	172	32	1012	48	5	81	4	1	20090510	2009-05-10T00:00:00.000+00:00	1	0.99	0.99	
9	188	34	1158	12	18	91	1	2	20090523	2009-05-23T00:00:00.000+00:00	1	0.99	0.99	
10														

10 rows | 1.77s runtime Refreshed 9 minutes ago

12. SCD Type 2 Demonstration

dim_customer implements Slowly Changing Dimension Type 2, which preserves full history of customer attribute changes. Three dedicated SCD2 columns are added to the dimension:

Column	Data Type	Purpose
effective_start_date	TIMESTAMP	When this version of the customer record became active
effective_end_date	DATE	9999-12-31 for current records; set to the close date when a newer version replaces this row
is_current	BOOLEAN	true = active current record; false = historical closed record

Change detection logic: A SHA-256 hash is computed over all 12 tracked attributes (FirstName, LastName, Company, Address, City, State, Country, PostalCode, Phone, Fax, Email, SupportRepId). On each pipeline run, the incoming hash is compared against the current active Gold record. If the hash differs, the old record is closed (is_current = false, effective_end_date = today) and a new active record is inserted (is_current = true, effective_end_date = 9999-12-31). An additional column row_hash stores the computed hash for auditability.

12.1 Version 2 Changes

To demonstrate SCD Type 2 behavior, three customer records were modified in Azure SQL (Version 2):

- Customer 1 (Luis Goncalves): City changed to Toronto, State to ON
- Customer 2 (Leonie Kohler): Email changed, Address updated
- Customer 3 (Francois Tremblay): Country changed to United States, City to New York

12.2 Version 2 Update in Azure SQL

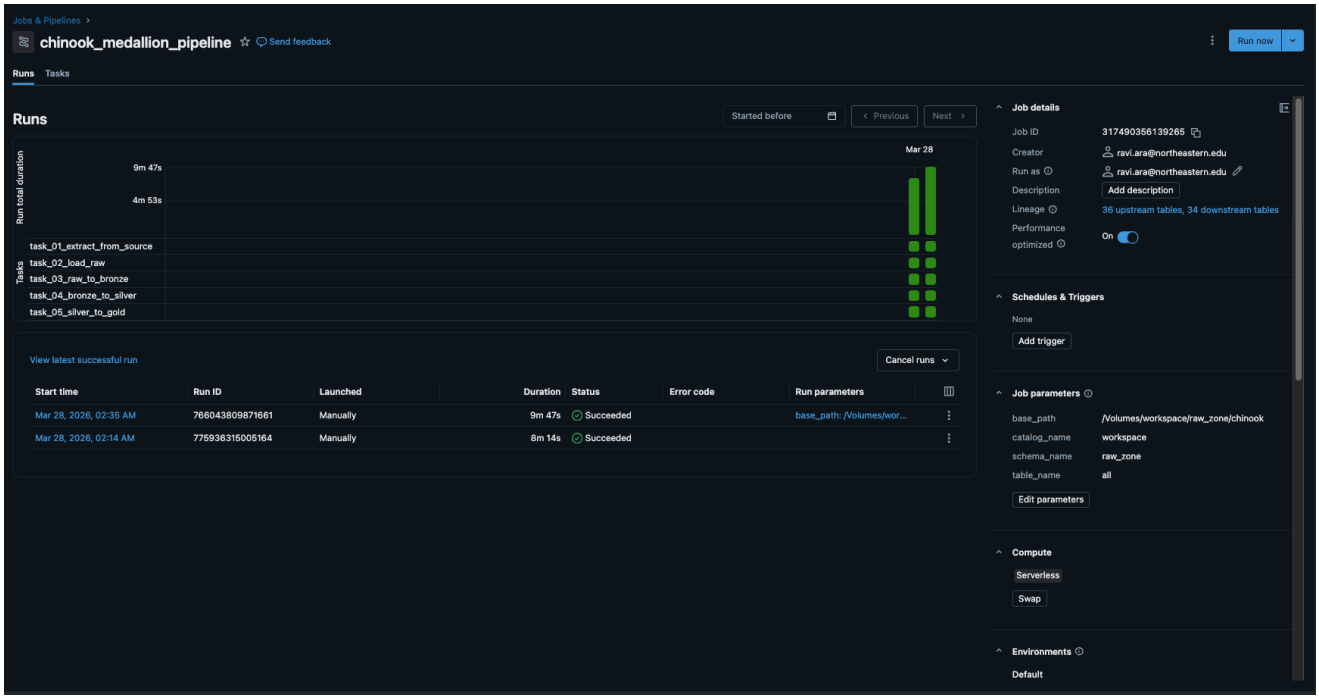
```sql
-- Version 2: Modify some customer records to trigger SCD Type 2
UPDATE chinook.Customer SET City = 'Toronto', State = 'ON' WHERE CustomerId = 1;
UPDATE chinook.Customer SET Email = 'leonie.new@surfeu.de', Address = 'New Address 123' WHERE CustomerId = 2;
UPDATE chinook.Customer SET Country = 'United States', City = 'New York' WHERE CustomerId = 3;
```

Messages Copy messages

2:34:32 AM Started executing on line 1
Query executed successfully

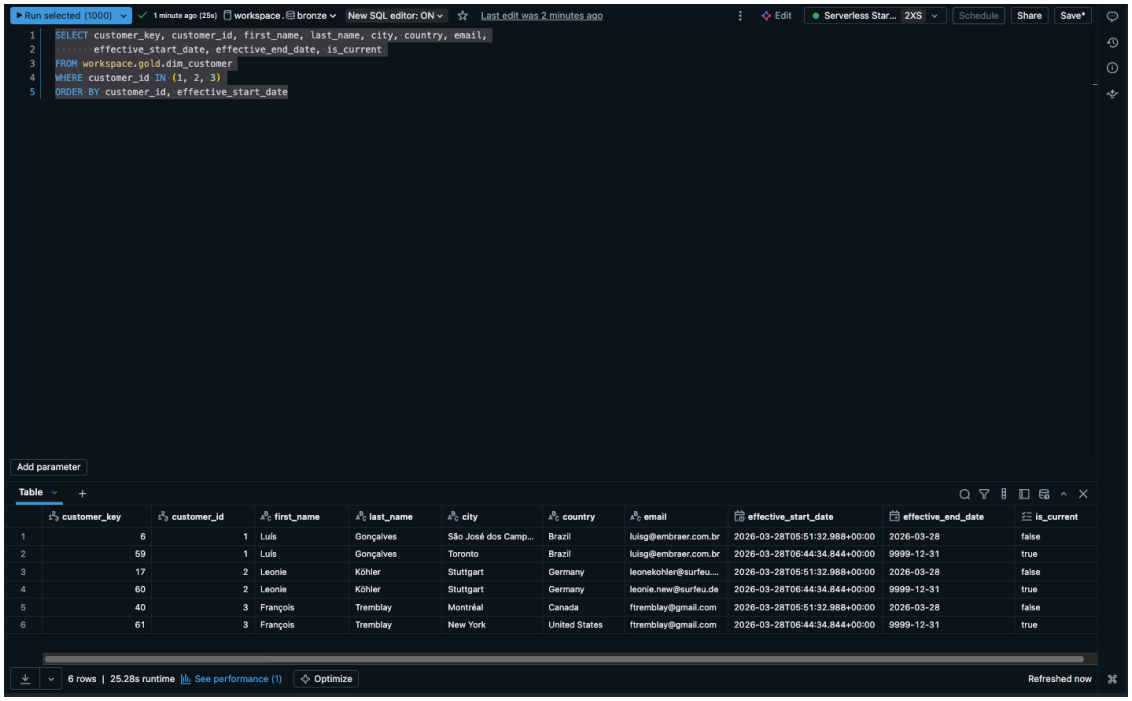
12.3 Pipeline Rerun (Version 2)

The pipeline was rerun via the Databricks Job. Second run succeeded in 9m 47s:



12.4 SCD Type 2 Result

Each modified customer now has exactly 2 rows: the old record closed (is_current=false, effective_end_date=2026-03-28) and a new active record (is_current=true, effective_end_date=9999-12-31):

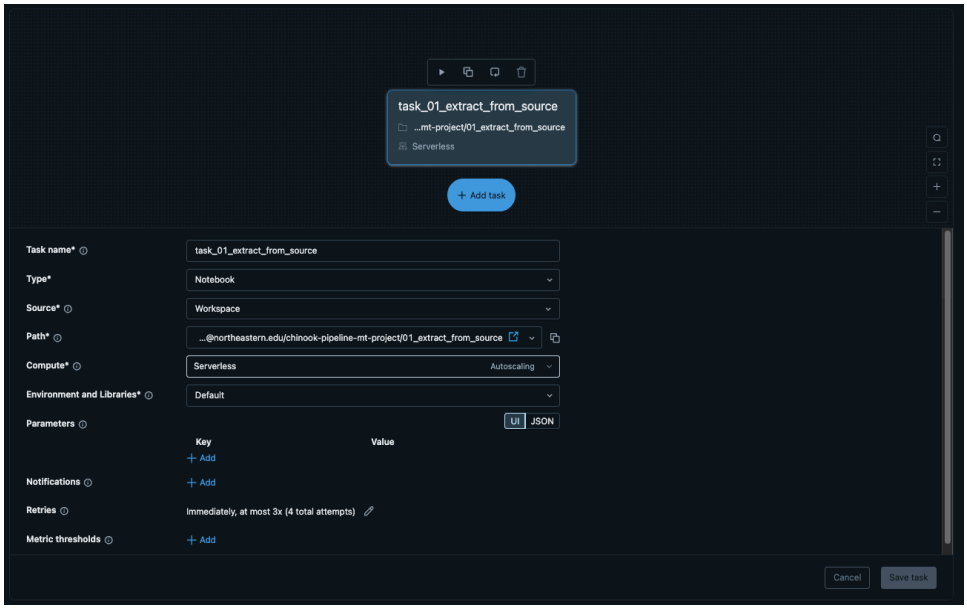


	customer_key	customer_id	first_name	last_name	city	country	email	effective_start_date	effective_end_date	is_current
1	6	1	Luís	Gonçalves	São José dos Camp...	Brazil	luisg@embraer.com.br	2026-03-28T05:51:32.988+00:00	2026-03-28	false
2	59	1	Luís	Gonçalves	Toronto	Brazil	luisg@embraer.com.br	2026-03-28T06:44:34.844+00:00	9999-12-31	true
3	17	2	Leonie	Köhler	Stuttgart	Germany	leonekohler@surfeu....	2026-03-28T05:51:32.988+00:00	2026-03-28	false
4	60	2	Leonie	Köhler	Stuttgart	Germany	leonie.new@surfeu.de	2026-03-28T06:44:34.844+00:00	9999-12-31	true
5	40	3	François	Tremblay	Montréal	Canada	ftremblay@gmail.com	2026-03-28T05:51:32.988+00:00	2026-03-28	false
6	61	3	François	Tremblay	New York	United States	ftremblay@gmail.com	2026-03-28T06:44:34.844+00:00	9999-12-31	true

13. Job Orchestration

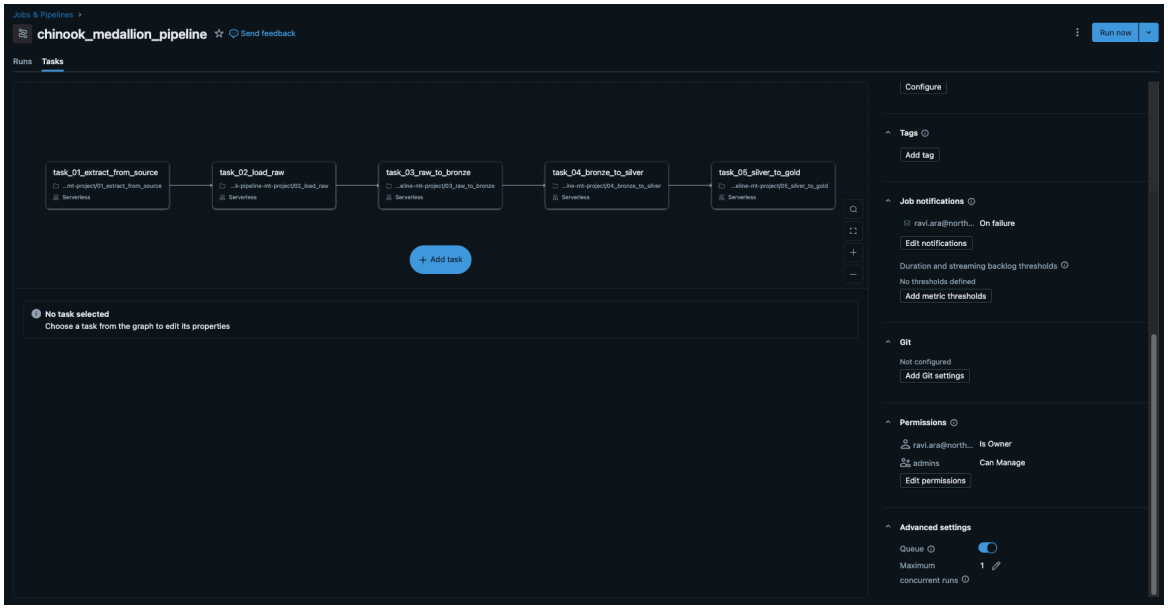
The full pipeline is orchestrated via a Databricks Job named chinook_medallion_pipeline with 5 linearly dependent tasks. Manual notebook execution is not required for production runs.

13.1 Task Configuration



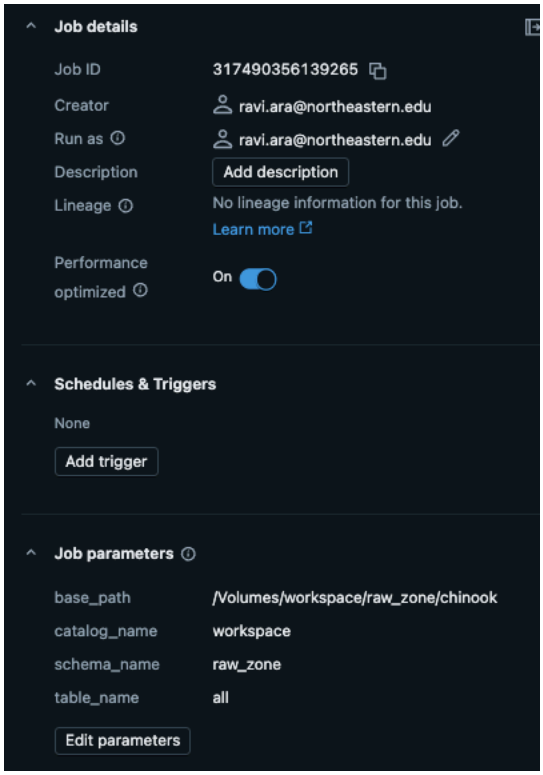
13.2 Job DAG & Configuration

Linear dependency chain: Extract → Load Raw → Raw to Bronze → Bronze to Silver → Silver to Gold:



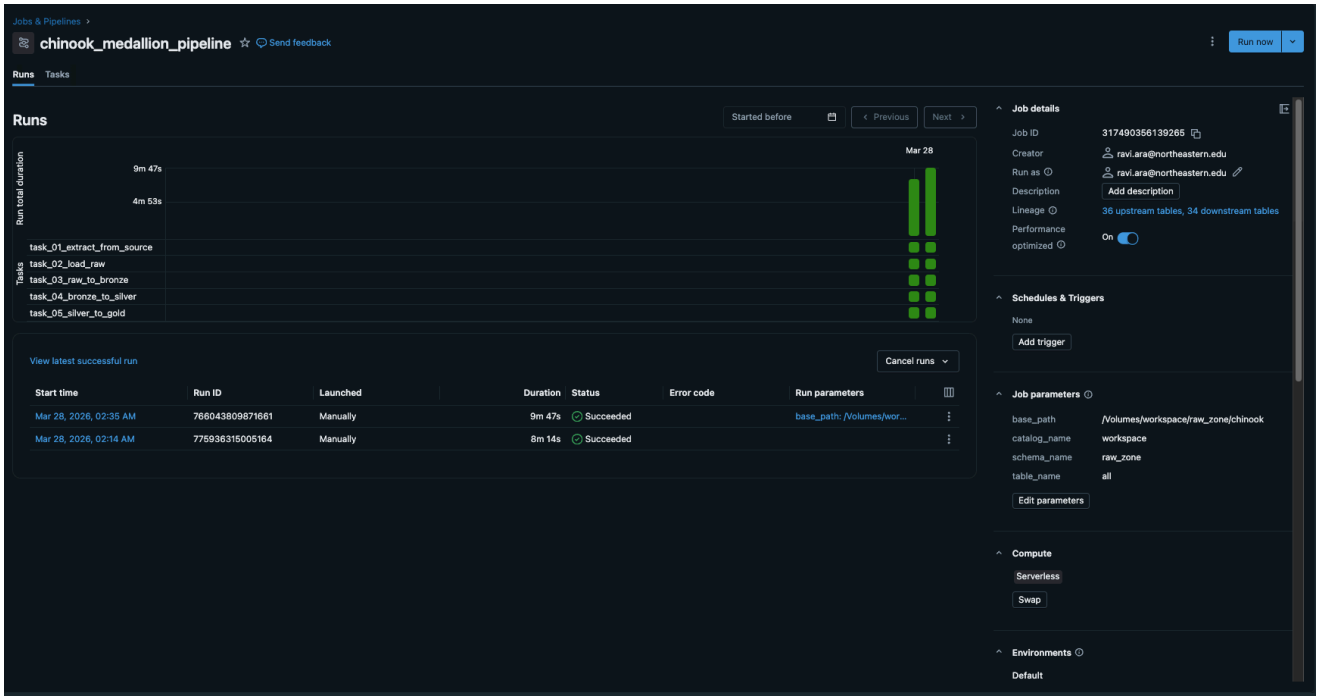
13.3 Job Parameters

All four required parameters configured at the job level, automatically pushed to all tasks:



13.4 Successful Run History

Two successful runs: Version 1 (8m 14s) and Version 2 with SCD changes (9m 47s). Email notifications configured for both success and failure:

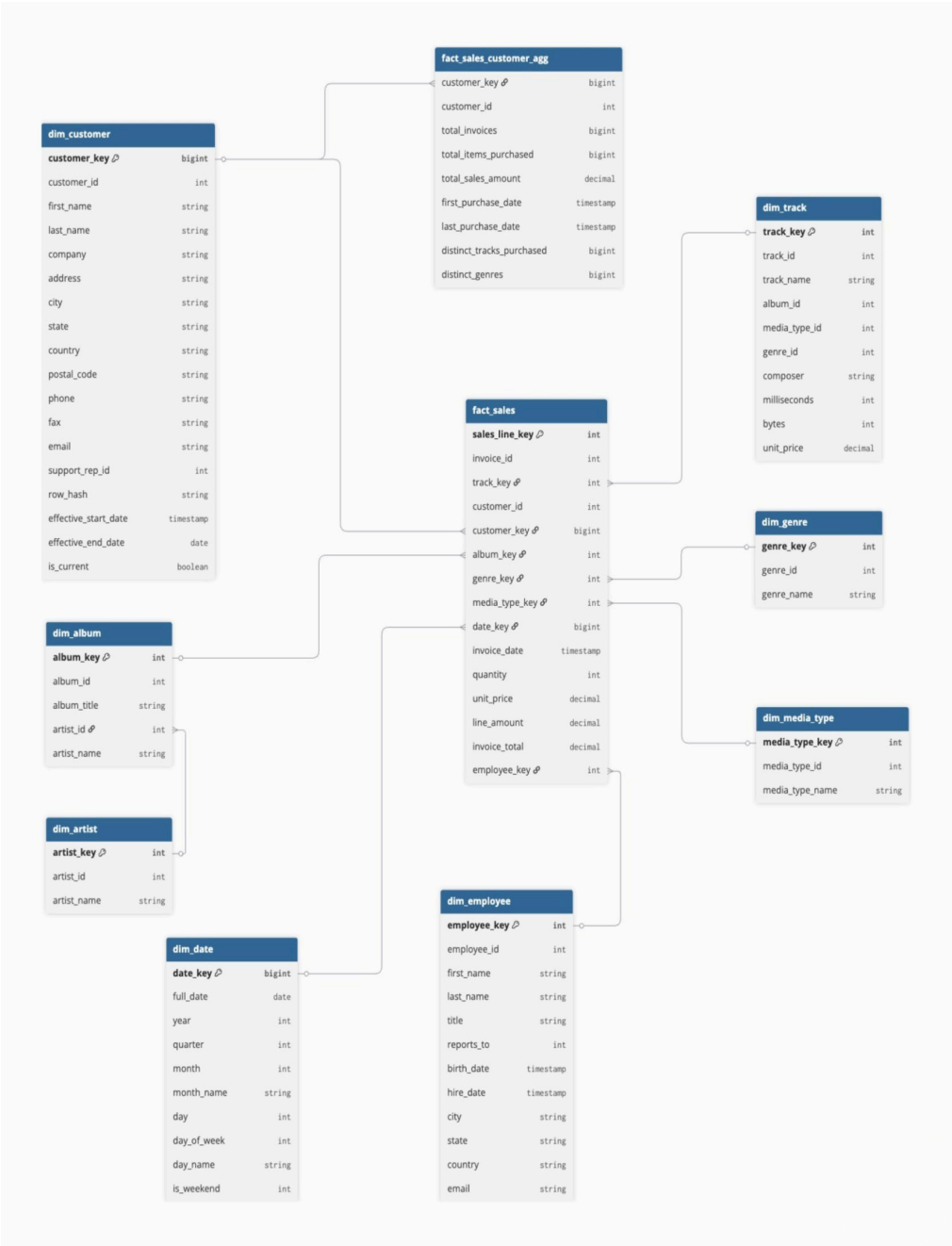


14. Dimensional Model

The Gold layer implements a star schema with fact_sales at the center, 8 dimension tables, and a downstream aggregate fact table. The model follows Kimball dimensional modeling methodology. dim_customer implements SCD Type 2 for tracking historical changes. dim_album links to dim_artist for the album-to-artist relationship. fact_sales_customer_agg is built exclusively from fact_sales (not from Silver) to maintain consistency with the dimensional model.

14.1 Star Schema ER Diagram

Generated using dbdiagram.io from the Gold layer DDL. All foreign key relationships, primary keys, and data types are shown:



14.2 Table Summary

Table	Description	Row Count
dim_customer	SCD Type 2 — customer demographics	59 (+ SCD rows)
dim_track	Track details: name, composer, duration	3,503
dim_album	Album with denormalized artist name	347
dim_artist	Artist reference (linked via dim_album)	275
dim_genre	Music genre	25
dim_media_type	Audio format	5
dim_employee	Support representative	8
dim_date	Calendar dimension (2009-2013)	1,826
fact_sales	Invoice line grain — quantity, price, amount	2,240
fact_sales_customer_agg	Customer-level aggregation from fact_sales	59

14.3 Key Relationships

Foreign Key	References
fact_sales.customer_key	dim_customer.customer_key
fact_sales.track_key	dim_track.track_key
fact_sales.album_key	dim_album.album_key
fact_sales.genre_key	dim_genre.genre_key
fact_sales.media_type_key	dim_media_type.media_type_key
fact_sales.date_key	dim_date.date_key
fact_sales.employee_key	dim_employee.employee_key
fact_sales_customer_agg.customer_key	dim_customer.customer_key
dim_album.artist_id	dim_artist.artist_key

15. Data Quality Rules

27 data quality rules were implemented across all 11 tables using Databricks Labs DQX (databricks-labs-dqx). Rules use DQRowRule with built-in check_funcs: is_not_null_and_not_empty for primary keys and required fields, and is_in_range for numeric boundary checks.

#	Table	Rule	Type
1	customer	CustomerId must not be null	Not Null
2	customer	Email must not be null	Not Null
3	customer	Email must contain @	Format Validation
4	customer	FirstName must not be null	Not Null
5	customer	LastName must not be null	Not Null
6	invoice	InvoiceId must not be null	Not Null
7	invoice	CustomerId must not be null	Not Null
8	invoice	Total must be >= 0	Valid Range
9	invoice	InvoiceDate must not be null	Not Null
10	invoiceline	InvoiceLineId must not be null	Not Null
11	invoiceline	Quantity must be > 0	Valid Range
12	invoiceline	UnitPrice must be >= 0	Valid Range
13	track	TrackId must not be null	Not Null
14	track	Milliseconds must be > 0	Valid Range
15	track	UnitPrice must be >= 0	Valid Range

Strategy: Quarantine + Logging. Failed records are written to workspace.quarantine.<table>. All rule executions are logged to workspace.metadata.dq_execution_log with total processed, passed count, and failed count.

16. SQL Scripts Reference

All validation and utility SQL scripts are included in the sql/ folder, organized by target platform:

16.1 Databricks SQL Editor

Script	Purpose
01_databricks_scd2_verification.sql	Proves SCD Type 2 — shows old/new rows for changed customers
02_databricks_gold_validation.sql	Gold table row counts, orphan checks, aggregate consistency
03_databricks_metadata_dq_validation.sql	Parent metadata, child metrics, DQ log queries
04_databricks_row_count_reconciliation.sql	Cross-layer count validation (source = bronze = silver)
05_databricks_silver_sample_queries.sql	Silver cleaning evidence (lowercase emails, trimmed strings)

16.2 Azure SQL Query Editor

Script	Purpose
06_azure_sql_version2_customer_updates.sql	Version 2 customer modifications for SCD Type 2 demo
07_azure_sql_gold_dimensional_model_ddl.sql	Portable DDL for data modeling tools (DBeaver, Navicat)
08_azure_sql_source_row_count_verification.sql	Source dataset completeness validation

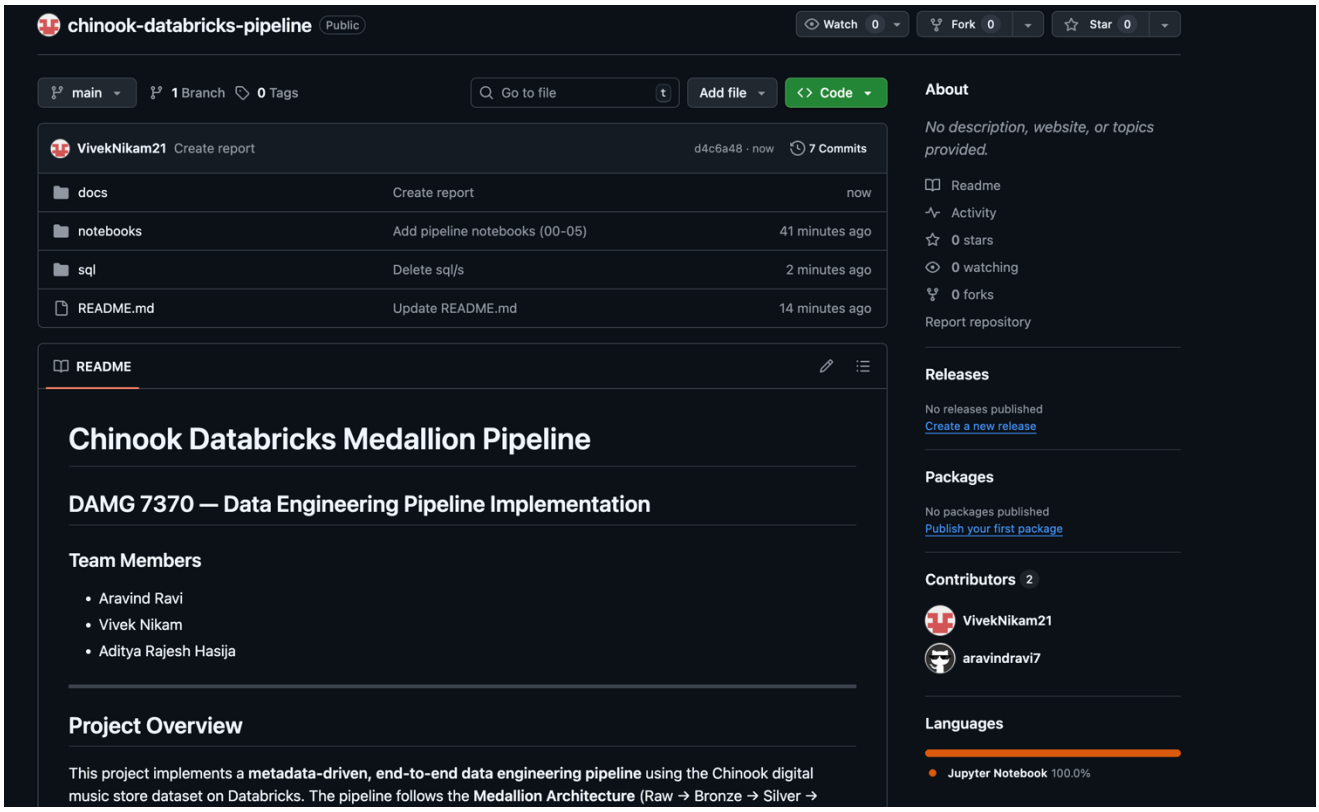
17. Validation Summary

Validation Check	Result
Source → Raw row count match	PASS — All 11 tables match exactly
Raw → Bronze row count match	PASS — All 11 tables match exactly
DQ rules executed and logged	PASS — 27 DQX rules, all logged
Quarantine mechanism operational	PASS — Schema and tables ready
Silver cleaning applied	PASS — Trim, lowercase, dedup
Gold dimensional model complete	PASS — 8 dims + 2 facts
dim_customer SCD Type 2	PASS — 3 customers show 2 versions
fact_sales grain correct	PASS — 2,240 rows (= InvoiceLine count)
Aggregate fact from fact_sales	PASS — Totals match: 2328.60
Orphan dimension keys	PASS — Zero orphans
Metadata-driven processing	PASS — No hardcoded table names
Parameterized notebooks	PASS — 4 widget params per notebook
Job orchestration	PASS — 5 tasks, linear deps, email alerts
Concurrency configured	PASS — Max 1 concurrent run
Dimensional model diagram	PASS — Star schema with all FK relationships

All validation checks passed. The pipeline is fully operational and production ready.

18. Git Repository

All project code, documentation, and artifacts are stored in a Git repository under the data_engineering_projects repository.

17.1 Repository Structure

```
chinook-databricks-pipeline
  notebooks/     (6 Databricks notebooks .py)
  sql/           (8 SQL scripts)
  docs/          (Mapping doc, documentation)
  README.md
```